Technology and Society
Computer Science

Matser's thesis

Radar data imitation for a visual docking guidance system

Khalil, Louay; Hojeij, Mohamed



Exam: Computer Science Master´s programme
Area: Computer Science
Supervisor: Magnus Krampell
Date: 2020-08-25

Abstract

Instead of being out at airports for the sake of testing radar data in Visual Docking Guiding System, one could use emulated radar data to function as real radar data. In this thesis, real radar data processed into binary files are, observed and dissembled to use reverse engineering and in such a way emulate similar binary files. The same software used on real radar data binary files is used to process the emulated ones to detect an object at a certain distance. Using reverse engineering could not on its own result in an emulated radar data binary file. Finally trial and error resulted in a file which gave object detection at a distance of above 21 meters as targeted in this thesis. Furthermore, as future work might be found by targeting other and further distances.

Sammanfattning

Istället för att vara ute på flygplatser för att testa radardata i Visual Docking Guiding System kan man använda emulerade radardata som kan fungera som verkliga radardata. I detta examensarbete observeras och demonteras verkliga radardata som bearbetas till binära filer för att använda reverse engineering för att skapa filer som efterliknar liknande binära filer. Samma programvara som används på riktiga radardata i binära filer används för att bearbeta de emulerade filerna för att upptäcka ett objekt på ett visst avstånd. Att använda reverse engineering var inte tillräckligt för en emulerad radardata-fil. Slutligen kunde trial and error resiultera till en binär fil som gav objektdetektering på ett avstånd av över 21 meter som var mål i detta examensarbete. Dessutom, framtida arbeten kan hittas genom att rikta sig mot ytterligare andra olika avstånd.

Acknowledgements

We would like to thank Magnus Krampell at Malmö University for all assistance during our research and for guiding us throughout our thesis. We would also like to thank Ulrik Ekdahl. Finally we would like to show our deep appreciation to Abdulla Al Chalati, a very dedicated engineer, for his patience with us and of course for his invaluable advice and continuous feedback.

Contents

1 Introduction **1**
- 1.1 Background . . . 1
- 1.2 Introduction . . . 2
- 1.3 Purpose and Motivation . . . 3
- 1.4 Research Hypothesis and Research Questions . . . 4

2 Theoretical Background **5**
- 2.1 Radar . . . 5
- 2.2 Chirp signals . . . 5
- 2.3 CPIs and binary files . . . 6

3 Literature review **8**
- 3.1 Survey . . . 8
- 3.2 History of Reverse Engineering . . . 8
- 3.3 Reverse Engineering and Design Recovery: A Taxonomy . . . 9
- 3.4 An Approach for Reverse Engineering of Relational Databases . . . 9
- 3.5 Model-Based Design to support Complex Systems implementation as a result of Reverse Engineering . . . 10

4 Research Methods **12**
- 4.1 Conceptual Framework . . . 12
- 4.2 System Architecture . . . 12
- 4.3 System Design . . . 13
- 4.4 Development . . . 14
- 4.5 Evaluation . . . 14

5 Results **15**
- 5.1 Construction of a conceptual framework . . . 15
- 5.2 Development of a system architecture . . . 16
- 5.3 System design . . . 17
  - 5.3.1 File beginning . . . 17
  - 5.3.2 Raw data: beginning, ending and size . . . 18
  - 5.3.3 CPI Production . . . 21
  - 5.3.4 Trial and error . . . 22
- 5.4 Development of the system . . . 25
- 5.5 Evaluation . . . 26

6 Discussion **27**
- 6.1 Iterative approach . . . 27
- 6.2 Results discussion . . . 27

7 Conclusion and future work **30**
- 7.1 Answering the research question . . . 30
- 7.2 Hypothesis . . . 30
- 7.3 Future work . . . 30

Appendix **37**

A Detections **37**

1 Introduction

1.1 Background

Airport is a place where aircraft take off and land. Each airport has at least one runway, where the aircraft takes off and lands. The buildings at major airports that facilitate the handling of passengers and goods are called terminals. From the terminal, passengers exit to their aircraft via a gate which is either simply a door to the plate where the aircraft is located (often there is a bus going out on the plate at larger airports), or an embarkation bridge, which docks with the aircraft [1] [2] [3] [4].

Going to an airport today is like going to a place with an invisible architecture with minimum interference of historical, cultural or society-belonging features [1]. With recent years of technological development, airports around the world are on the lookout to increase efficiency and safety. The productivity of an airport has been a focus for a large number of studies [4]. Airports are implementing smart solutions for operational and economic effectiveness. Many airports find it crucial to shorten the aircraft turnaround time [5]. At the same time, airports and airlines are striving to minimize the risk of accidents in the apron area where there is a constant flow of aircraft, ground vehicles and people. Figure 1 illustrates an apron area.

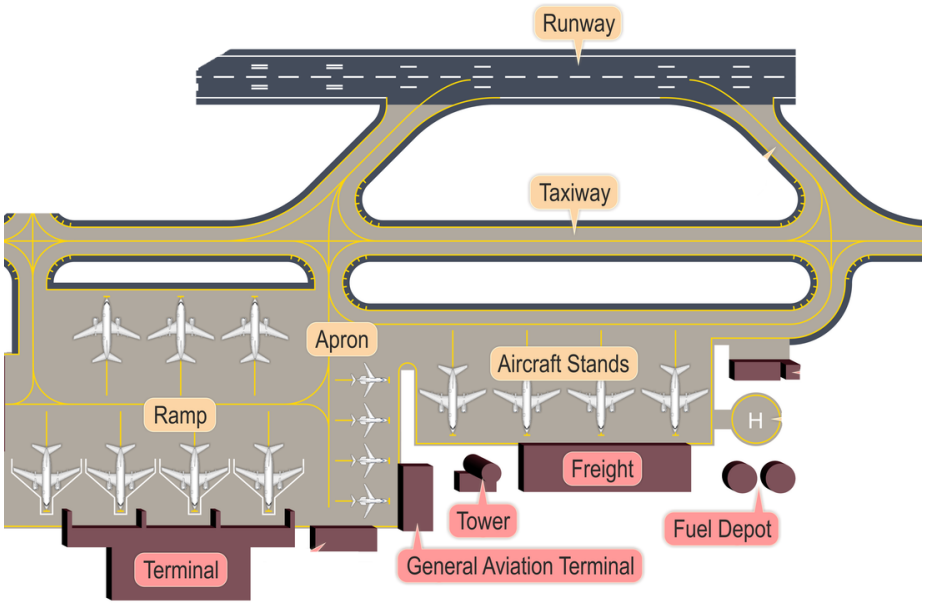


Figure 1: Illustration of the outside-area of an airport [6], wikipedia.org

A company in Sweden is currently dominating the market with products for gates, airfields, towers, several airport service and even airport management is

called client in this thesis report. The company has come along way developing products for gates. In their vision of shortening the turnaround time they are constantly searching for the better options. Already in place, at airports with products from the client, outside the gates are pilot-guiding-displays called VDGS(Visual Docking Guidance System). After landing, the aircraft is parked at a docking station to connect to a boarding bridge. The two most crucial functions of the VDGS: the first is when possible to guide the pilot to a straight line through the apron area when parking the aeroplane, while the other function is to measure the length of time during which an aeroplane stays parked outside a gate. Airlines get charged for the length of time their aeroplanes stay parked outside each gate. Also, when lowering the time aeroplanes are on the ground to a minimum, airports can to the full extent utilize their capacity and more safely be able to welcome more traffic without changing their facilities [5].

VDGSs [7] [8] [9] function differently depending on the manufacturer or the main purpose but in general, they are there to guide the aircraft to stop at the right spot, counting the time of parking the aeroplanes as well as detecting obstacles around. Some of them use 3d cameras, LIDAR, and even radar to detect objects in a particular area.

Until now, the client uses LIDAR (Light detection and ranging) with laser light in their VDGS systems. The LIDAR can scan and detect aeroplanes from far away, all the way to a gate. By finding the fastest way from landing to gate and out of the gate, the number of aircraft waiting in line on ground or in circles in air will be decreased.

1.2 Introduction

The key factor for successful airport operations is real-time management [10]. In low visibility conditions, real-time management is disrupted, especially when a VDGS cannot determine if an aircraft parks at the gate or not. Also, the turnaround time is affected because of that. An example of low visibility condition at an airport is seen in Figure 2. In that fog and that state of no visibility, the use of LIDAR or cameras is useless. The client plan to use a radar sensor for those situations. They are working on producing a final product(radar system) to communicate with a VDGS. A radar system can be used to detect aeroplanes, vehicles and other objects. A challenge facing the client in their developing process of the radar system is how they can verify that a VDGS(Software) can handle its functions with the addition of a radar system next to the LIDAR. For the LIDAR, they already have a laser-simulator-software to test the functions of the VDGS. For the radar system, they want to have something to replace the real radar system for testing purposes.

Figure 2: Picture of an airplane hidden in thick fog at an airport

1.3 Purpose and Motivation

The purpose of the work is to build a radar emulation tool, that can generate data similar to that of a real radar system. Using a radar sensor with high frequency is improving detection of objects as small as 10 cm by 10 cm in most weather conditions [16]. Data about these objects is sent to the VDGS. Depending on what data is sent, the VDGS acts differently. There is a need for a testing tool for such VDGS. The objective of this this research is to prepare the data for such testing tool which can be connected to a VDGS simulator to perform testings given that the prepared data appear to be as real radar data. It is important because it realize testing without the inconvenience of being out at airports. We want to make testing accurate and simply available at any office. The later on proposed radar emulation tool will upon request pre-generate binary files to be sent to a processing software developed by the client. There after the emulation tool shall receive results from processing the generated binary files. In Figure 3, an example of detections from real radar data is shown where the x-position of the detected object is the most important metric because it identifies the distance from the radar and straight out to the object. The client perform their software tests where parked airplanes are parked at a distance of above 21 meters from the wall of the gate-building.

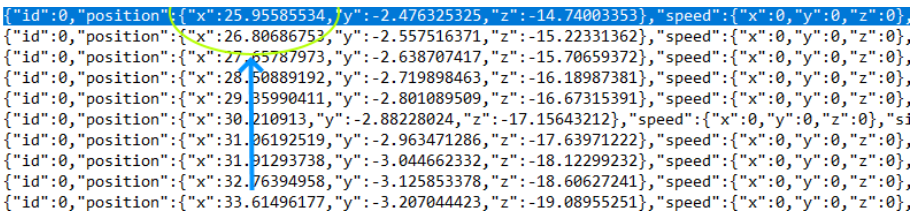


Figure 3: An example of detection as a result of processing radar data

1.4 Research Hypothesis and Research Questions

The hypothesis for this thesis

Hypothesis: It is possible to generate data so that a VDGS system thinks it is real radar data.

Any data that comes with detailed documentation and detailed structure manual should in many cases have the ability of being replicated. In those cases it would not be difficult to prove the hypothesis. The proposed hypothesis is challenged with the lack of real radar data documentation. As mentioned further in the methodology chapter and the result chapter, the real radar data is provided for this research in binary files without any knowledge about the content inside. The proposed hypothesis can be proven by finding relationships between variables [12] in those binary files using Reverse Engineering. The outcome of this thesis is considered as a proof of concept in the shape of a radar emulation tool and the research question is:

RQ: How can Reverse Engineering of real radar data, help to generate imitated data and function as if it was real radar data?

2 Theoretical Background

2.1 Radar

The radar is not a new technology and goes back to the early 20th century when it was first patented and demonstrated by a German engineer in 1904. Later on, radar was first demonstrated in Britain the summer in 1935 [20]. In short, the radar detects an object by transmitting an electromagnetic (EM) wave and then analysing the wave echo that comes back from the object. From that echo range and velocity can be extracted. The waves are transmitted different frequencies depending on the purpose of the radar system [21]. The range of detection or the the distance to an object connected to either low or high frequency. Short Range requires High frequency while low frequencies are preferably used for long-range. To calculate the range, there is the classic radar range equation in Equation (1) [18] [20] [21] [22] [23].

$$R = \sqrt[4]{\frac{P_t G^2 \lambda^2 \sigma}{P_r (4\pi)^3}} \tag{1}$$

In the equation above:

- R = Range
- P_t = Transmit signal Power
- G = Antenna Gain
- λ = Transmit wavelength
- σ = Target radar cross section
- P_r = Reflected signal power

2.2 Chirp signals

Chirps are pulses which start with one frequency and then either increases with time in cases of up-chirps or decreases in down-chirps [24] as illustrated in Figure 4.

A radar system works as follow [25]: when energy of a radio frequency is discharged from an antenna, some of that energy is reflected from objects back to a receiver near the transmitter Because radio waves travel with a constant rate and with the help of the time between transmitted and received signal, the distance to an object or target is calculated. Different lengths give different radio burst lengths such an example as 1-microsecond pulse will be provide a 300 meters long radio burst. That give a resolution of about 300 meter length. Better resolution requires shorter pulse. On the other hand to detect objects much further would require more energy. That conflicts with the requirement of shorter pulse for better resolution. Chirp signals are used in an ingenious

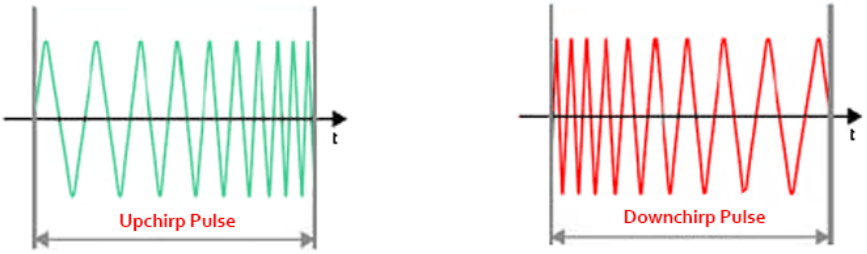


Figure 4: Illustration of one down-chirp and one up-chirp, from [28]

way to implement stable echolocation systems to tackle those two requirements. Echolocation [26] [27] originate from animals specially the microbats and how they send out loud sound with short pulses to be able to locate things in total darkness. They vary their pulses depending on the hunting strategies they follow. Not only that, also they use the combination of: the time delay from the discharge and reflection of the echo, the difference in time between their two ears, and finally the variations in loudness of the echoes. [25].

2.3 CPIs and binary files

Analogue-to-Digital Converter known as ADC takes samples with different time intervals of the analog signal to be converted to digital form [22]. Basically, ADC is a device used to convert an analog input current or voltage to a digital number which is proportional to the magnitude [31].

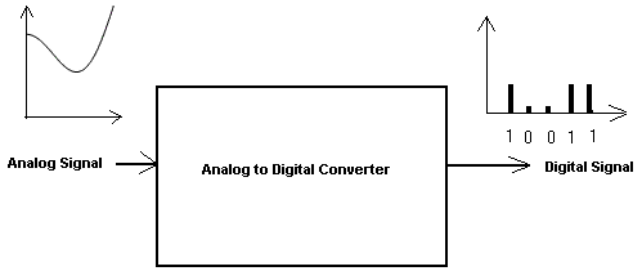


Figure 5: Illustration of a signal in a Analog to Digital Converter, from [31]

In Figure 5, an analog signal is represented in a graph before getting converted to a digital form. Texas Instruments mention in [32] that captured data containing CPIs(Coherent Processing Interval) is stored in binary files where the values can

be shown in different formats like binary, hexadecimal and others. According to [5], every CPI contains 512 chirps where the data is stored in a binary file with an exact size of 3 154 944 *bytes*.

3 Literature review

In this chapter, Reverse Engineering is explored in a brief manner and some recent researches involving it is reviewed. The review of related works is of help as guidelines for the process of Reverse Engineering in this research.

3.1 Survey

Reverse Engineering is found [33] [34] [35] [36] [37] [38] [39] [40] [41] [42] [43] [44] [45] [46] [47] [48] [49] [50] to be applicable in multiple and different fields of engineering whereas for software and binary reverse engineering the three main types are:

- Observe and analyze
- Disassemble
- Decompile

In the field of software the two main objectives [34] are redocumentation and design recovery. Further more, when the source code of a target software is missing [48] [49] [51] [52], and when analysis is done by observing exchanged data [53] [54] [55] the two types of reverse engineering are:

1. Binary Reverse Engineering
2. Protocol Reverse Engineering

The packet format-analysis started as a manual process but some automatic processes have been proposed by researchers [53] [54] [55] [56] where the main automatic types are:

- using various clustering analyse
- emulate the protocol implementation tracing the message processing

3.2 History of Reverse Engineering

Reverse Engineering is not specially new to engineers and is used for both hardware and software as it is found when searching about this subject. In 1985, a senior member of the IEEE M. G. Rekoff JR, released a paper [33] where Reverse Engineering has been successfully applied to a complete larger weapon system and parts of others. Public and unclassified archives from Red Stone Information Center, Huntsville, AL, failed to disclose any previously published literature on Reverse Engineering methods as defined in that paper [33]. Rekoff considers Reverse Engineering as the process of developing a set of specifications for a complex system through an orderly examination of sections of that system. These specifications are prepared by other than the original designers, without taking advantage of the original design or other documentation. It is assumed that the specifications are developed to create a clone of the original system,

a circumstance that requires the most comprehensive form of specifications according to Rekoff.

Public and unclassified archives from Red Stone Information Center, Huntsville, AL, failed to disclose any previously published literature on Reverse Engineering methods as defined in that paper [33]; consequently, no previous work has been cited.

3.3 Reverse Engineering and Design Recovery: A Taxonomy

In [34], Reverse engineering is explained as the process of analyzing a system to identify the building blocks in it and to understand their interrelationships for a creation of a representation of that system in another shape or at a more complex level of abstraction. Further on in [34], Reverse Engineering is outlined with multiple stages. Stages that are considered to be important are:

- Explore available implementation of the system
- Recapture/recreate the design of the system
- Decipher implemented requirements by the system

It is also mentioned in [34] about multiple subareas of Reverse Engineering and the two of them which are often referred to are:

- Re-documentation
- Design recovery

3.4 An Approach for Reverse Engineering of Relational Databases

Usually, there is no obvious, single correct answer for a design of the process of Reverse Engineering. Multiple interpretations can yield plausible results. The process in this paper [36] emphasizes analysis of database keys. The recommended process for Reverse Engineering is summarized with the following steps to be applied on databases:

- Prepare an initial object model
- Determine candidate keys
- Determine foreign key groups
- Refine tentative classes
- Discover generalizations
- Discover associations
- Perform transformations

Researchers conclude in [36]:

- Designers, even the experts, sometimes violate rules of good design and often employ unusual constructs. In some cases, it is impossible to produce a complete, accurate model of the design.
- A flexible, interactive approach to Reverse Engineering is more likely to succeed. The designer needs a suite of flexible, loosely coupled tools.
- Determination of candidate keys and foreign keys is an important step in Reverse Engineering. Finding candidate keys is easy. Finding foreign keys is difficult.
- It is usually possible to look at the schema as whole and deduce the underlying strategy. Re-engineer-tools should allow the user to specify assumptions to guide the Reverse Engineering process.
- Transformations are useful for exploring the space of equivalent (equivalent given the context of semantic insight) models and approaching the original intent of the designer.

3.5 Model-Based Design to support Complex Systems implementation as a result of Reverse Engineering

Reverse Engineering is notably used in software engineering [34] and other scientifically inspired engineering design [37][38], making use of the core principles presented in Rekoff's work On Reverse Engineering [33]. Along with this root conceptual work, various contemporary works [39][40][41] give information on multiple Reverse Engineering techniques.

The first step in Reverse Engineering in [42] is planning. Regardless of the method followed or documentation used, is a plan necessary to maintain the organization and prevent or mitigate mistake. Next is disassembly - one of the most important steps in understanding the function of the product, and therefore in selecting models to represent its function. While Rekoff [33] suggests acquiring a number of identical copies of the product to be disassembled, Otto and Wood [39][40] opt for a more methodical disassembly process which relies on documentation and precise cataloging of each component to gain full component knowledge from a single example of the product.

Otto and Woods' [39][40] approach is popular in mechanical design, where components' functions and physical layout are closely related, while Rekoff's [33] more cautious approach can be advantageous when disassembling highly complex systems. Keeping these differences in mind, aspects of both approaches are applied in [42]. Since these methods differ so drastically, it is advisable to apply the steps which are best suited to the specific case. A large part of Reverse Engineering is the creation and estimation of "barriers" - design elements which expressly inhibit Reverse Engineering. The goal of the product's design team

is to inexpensively create barriers which will likely make Reverse Engineering more expensive to the competitor than simply designing a new product.

4 Research Methods

The five steps, seen in Figure 6, in the Nunamaker systems development process or SD, are set up for the sake of building a prototype. Step number four is called "Build prototype" and is aimed for building a prototype. Looking at the other steps, they are necessary to succeed with the building of the prototype where it is possible to iterate between these steps for the sake of system improvement. This method is chosen to build a prototype following the five steps of the method because of the iterative and systematic approach found in the five steps in Figure 6. With such an approach, a concept can transform into a prototype [63].

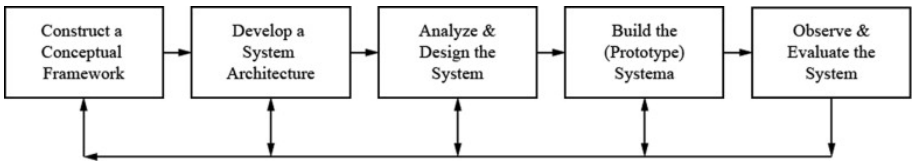


Figure 6: Nunamaker Systems development five steps [66]

4.1 Conceptual Framework

The first step in the Nunamaker method (based on Nunamaker et al 1990-1991) [63], is Concept Building: the definition of the problem area, and understanding and identifying the problem with enough knowledge about it. The construction of a meaningful research question is the inevitable ending of the introduction chapter before going on to investigating the functionality of the system and studying up to date available researches, for the subject in this thesis. Further more, the literature review of related works in Chapter 3 shall make it possible to recognize a gap that could be filled when attempting to deliver something new to the field of the research problem. Also, the review of Reverse Engineering-papers starting from the early origins up to recent works has a purpose of scientifically understand Reverse Engineering and extract guidelines during the third step in Nunamaker: System Design.

Information is taken from academic databases and scientific sources. Another part of the conceptual framework is to conduct interviews with engineers from the client to obtain requirements upon which an evaluation of the prototype is done. The major part of the conceptual framework is a review of papers about Reverse Engineering found in Chapter 3 where it lead us to the Binary Software Reverse Engineering using captured data to observe and analyze.

4.2 System Architecture

Next to do is to develop a system architecture. A good system architecture gives a journey plan for the system build-up process. It sets the system components in a correct perspective, system states functions and defines structure relationships and dynamic interacting between system components, in order to act as a guide

during the building process of the radar emulation tool. In the development type of the research, researchers must state the development objectives translated as the focus in the research and finally define the functionality for the resultant system in order to achieve the stated objectives [63]. Therefore, the requirements should be outlined in a way to be measurable. That would be of great help when reaching the validation stage in this research. A system architecture shows what kind of prototype is built, which is seen in next chapter. An architecture, physical and logical, shall include the components used to build the prototype.

4.3 System Design

Design is an essential part of a prototype development process. Design means understanding the studied domain, application of relevant scientific and technical knowledge. The prototype is designed with its essential parts that are necessary to fulfill the requirements of the prototype functionality. Design specifications are used as a blueprint for the implementation of the system within the prototype. The program module and the functions should also be specified at this time including both hardware and software. The design is illustrated in multiple diagrams and is supposed to show how the prototype will be built [63] by performing reverse engineering on binary files. The Reverse Engineering of the radar data requires real radar data which shall be ready and at hand.

Each binary file contains exactly 3154944 *bytes* and with A HEX-editor software is used to find relations within the structure of these bytes. Then we start a process of breaking down these bytes in smaller and smaller sections as part of the clustering analyse from 3.1. Noted, that clustering analyse is typically used with available data protocol specifications, but the given binary files do not have such specifications.

The first step is to breakdown our architecture into parts or components such as:

Prepare initial object model

- File size
- File type

Determine essential values:

- File beginning
- Raw data beginning
- Raw data ending
- Raw data size
- File ending

Discover generalizations

- Repeated values
- Shared values
- Useless values (values to be ignored)

Discover associations

- Compared with detections
- Compared with failed processing

EU Directive 2009/24 on the legal protection of computer programs, which superseded an earlier (1991) directive,[43] governs reverse engineering in the European Union.[44][45]

With a successful design recovery the next step is to replicate data as per the design in the next stage of Nunamaker, the Development stage.

4.4 Development

Implementation of a system is used to demonstrate the feasibility of the design and the usefulness of the functionality in a system development research project at any time. The process to implement one working prototype is much so dependable on the design mentioned in 4.3. Empirical studies of functionality and usefulness can only be carried out after that the prototype has been built and is ready. In case of facing obstacles or coming across some problems that the design may lead to, the iterative approach in Nunamaker makes it possible to go back to the foregoing stages and re-design or make necessary adjustments [63]. Reverse Engineering of real radar data is performed to achieve highest possible rate of success in consideration and dependence of available resources. The resources are such as: literature review, documentation of the available real radar data and other information from the client.

The development stage also includes the connection between various components in accordance to the the system architecture from 4.2. The success rate of replicating radar data depends on how thoroughly the Reverse Engineering is performed on the real radar data.

4.5 Evaluation

The prototype is evaluated in this step with the possibility of adjustments through an iterative approach. The ultimate evaluation of the emulation tool is done by having ready replicated CPI:s and send them to the client processing software to finally receive detection results of a distance above 21 meters which is the shortest distance between an aeroplane nose and the wall on which the radar visual guiding system is installed. This distance is decided by the client. Results from a test of processing reverse engineered CPIs into JSON-files is interpreted and evaluated on the basis of its conceptual framework [63].

5 Results

In this chapter, results achieved in this research are presented. In particular, these results are kept at an objective level without any personal inputs and divided in subsections representing the subsections from the methodology chapter.

5.1 Construction of a conceptual framework

The conceptual framework started already in Chapter 1 by introducing the research problem with a background on airports and aircraft parking followed by an introduction of the objective of this research where the purpose and the motivation points that a prototype of a radar emulation tool is built as a proof of concept for the hypothesis in 1.4. Since this research is about radar data, the theoretical background chapter briefly introduced the usage of radars and their signals processed to data.

Information is primarily taken from databases like IEEE Xplore Digital library, Science Direct, and ACM Digital library. Using various resouces for the components of our proposed structure of Reverse Engineering requires that they do not contradict one or and another. On the other hand it is necessary to choose components compatible to one and another in the perspective of being able to interact and be functional with each other.

Other part from the information used in our research is obtained through interviews with employees working for the client as well as brief radar data documentation that they could not be reveald in this thesis because of No Disclosure Agreement. A software developer responsible for radar VDGS simulator communication pointed out that the detections which were obtained as a result of processing CPI:s shall be delivered as a JSON-file. The setup or rather the design of the JSON-file is found in 5.3.

- REQ1: Object detections in JSON-file.

The engineer responsible for the radar sensor programming part pointed out that the radar system performs five scans per the second independent from how many scans the VDGS can receive.

- REQ2: Five detections/ Five JSON-files per second.

A senior software architect pointed out that all their existing test tools and simulators are running on windows and windows server and any new software like our prototype should do the same but the software for processing the CPI:s runs on Linux.

- REQ3: Server/client communication: prototype as client.

The software developer of the VDGS simulator pointed out that it can only receive responses such as parking spot is busy or empty. Also, the VDGS simulator will in the future expand with more functions such as receiving responses

about moving objects and multiple objects which will be discussed in the future work section.

- REQ4: Detection of a distance of above 21 meters

5.2 Development of a system architecture

A detection is a detection of an object, in the case of this thesis aeroplane, at a certain distance from a given starting point. Detections are represented as measurements of that distance of the object. The two main functions/components of the system as seen in Figure 7 are a requested distance and detected distance in JSON-format. The figure also illustrates the sub-functions of the system.

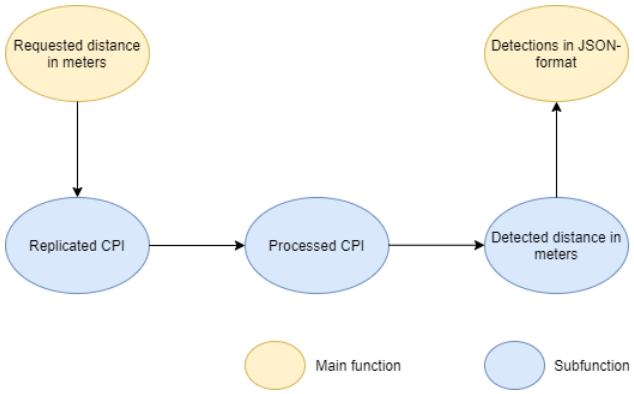


Figure 7: Simplified logical system architecture

A logical view is done here as seen in Figure 8 to illustrate the system in three major parts: Required for required software, Already existing for the client CPI processing software and the communication part in between.

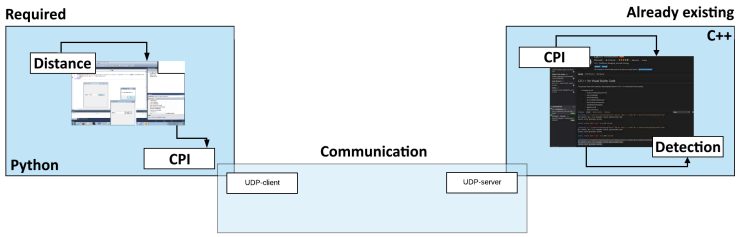


Figure 8: Simplified system architecture

5.3 System design

The main component required as seen in the architecture in 5.2, in Figure 8, is a CPI matching a certain distance. The client is providing a CPI processing software which dictates the CPI to be in binary format (.bin files) with a size of 3154944 *bytes*. Furthermore, 2764 CPIs, binary files, matching real radar data captured at an airport are provided without file or data design specifications.

The proposed system design n this section aims to identify or recover the design of data/CPI:s provided by the client. Data for one single CPI is delivered in one binary file without any information or instructions as to what they represent. That means that each one of them needs to be sent for processing to obtain a result of detections. A very brief summary was provided of the setup of the bytes inside a binary file by the client but this summary was very abstract. Reverse engineering is performed on the provided binary files to produce a design upon which the replicated CPIs are built when replicated.

The Reverse Engineering is designed in stages and sub stages, going from one stage to another with the possibility of iterating:

- Identification of a token, a sequence of bytes, for the File Beginning Key and the File Ending Key.
- Identification of a token for the Raw Data Beginning Key and the ending key.
- Identification of various sizes of raw data.

5.3.1 File beginning

With a manual search, using the HEX-editor, on 50 randomly chosen files of all 2764 available binary files a sequence of bytes was found to at the beginning of those files.

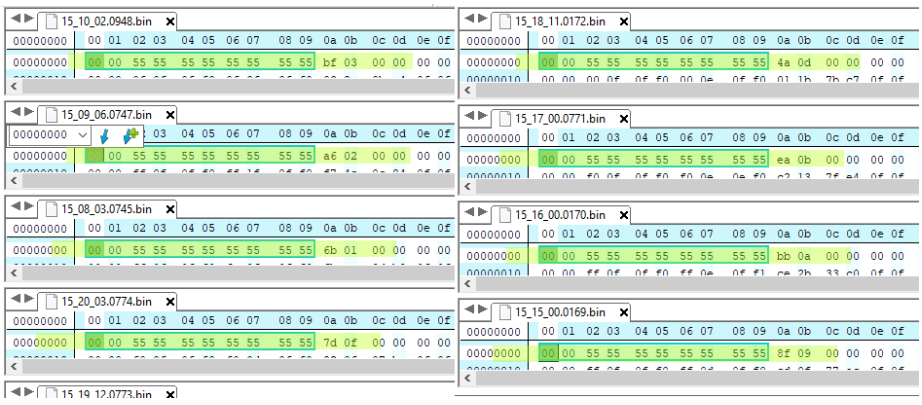


Figure 9: Screenshot from Hex-editor manual search in binary files.

A simple Python software at this stage was developed for looping through all 2764 available binary files: the sequence of bytes of "00 00 55 55 55 55 55 55 55 55" was found at the beginning of every single file. Also, this sequence was never repeated at all anywhere else.

- Byte-sequence: "00 00 55 55 55 55 55 55 55 55" for file beginning

5.3.2 Raw data: beginning, ending and size

While the files do not have any obvious or certain token for the ending we found that the sequence of "00 00 83 83 83 83 83 83 83 83" was repeated 6162 times in each file. The client pointed out that the bytes inside a binary file are either divided on 512 or 256 *chirps*. To find out, we divide 3154944 *bytes* by 512 *chirps* to 6162 *bytes/chirp*. The result was that a CPI contains 512 *chirps*.

- Byte-sequence: "00 00 83 83 83 83 83 83 83 83" as a Chirp Start
- 512 *chirps* á 6162 *bytes*

Hereafter, another re-occurring byte-sequence was found which comes straight after the Chirp Start sequence acting as a counter for all the CPI:s represented as binary files, in other word a CPI number. The sequence was made of 4 fragments where the last 2 of them in all files are "00 00". The binary files are provided by the client sorted by the time stamp for when the data was collected. Looping through those files following that order, a significant association was found between the first two fragments and all the 2764 provided files. Every fragment represents one byte and can be displayed in different forms such as HEX and decimal where the highest decimal value of a byte was 255. Figure 10 shows the binary content of two files where the time stamp was highlighted. Also, the decimal values of the two first fragments are highlighted pointing out how the first fragment was on 255 and in the following file according to the time stamp starts at 0 and how the second fragment increased from 9 to 10.

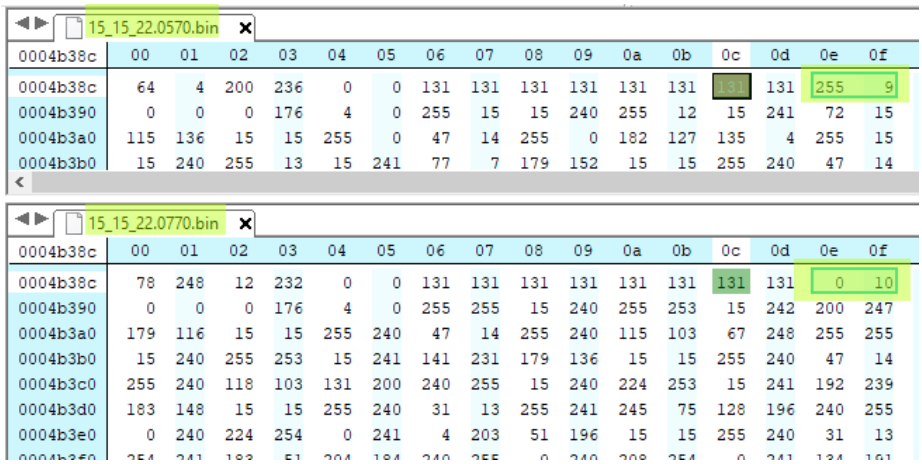


Figure 10: Screenshot from Hex-editor manual search in binary files.

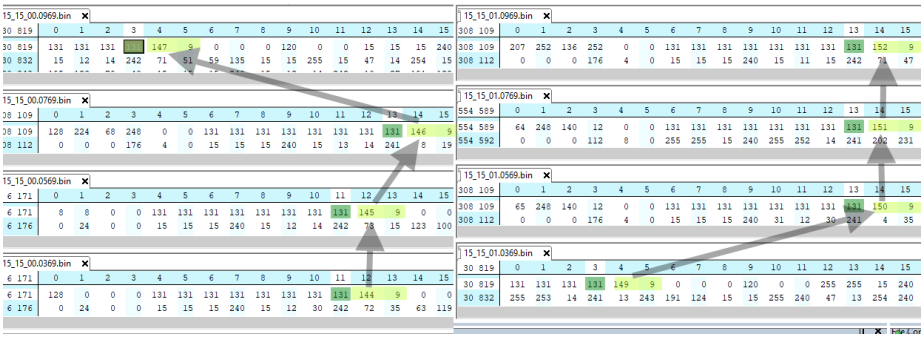


Figure 11: Screenshot from Hex-editor manual search in binary files.

There was a CPI number after each Chirp start token which was unique for each CPI/binary file in all of the 2764 provided files. FF was the highest HEX-value of one byte/fragment as seen below.

- Byte-sequence: "FF FF 00 00" after the Chirp Start as a CPI Number

Again, a byte-sequence of 4 fragments was found re-occurring in a certain way after each CPI Number. The byte-sequence increases for each new Chirp by exactly the same value. The first Chirp in all files shows that sequence as the following:

- Byte-sequence: "00 18 00 00"

The byte sequence in HEX-format in the following Chirps in the same file are increasing like the following:

- Byte-sequence: "00 30 00 00"

- Byte-sequence: "00 48 00 00"
- Byte-sequence: "00 60 00 00"
- Byte-sequence: "00 78 00 00"
- Byte-sequence: "00 90 00 00"
- Byte-sequence: "00 A8 00 00"

The byte sequence in Decimal-format in the following Chirps in the same file are increasing like the following:

- Byte-sequence: "00 30 00 00"
- Byte-sequence: "00 48 00 00"
- Byte-sequence: "00 60 00 00"
- Byte-sequence: "00 78 00 00"
- Byte-sequence: "00 90 00 00"
- Byte-sequence: "00 A8 00 00"

Addition of two HEX-values 00180000 + 00180000 results into a HEX-value of 00300000, 00300000 + 00180000 results into 00480000 and 00480000 + 00180000 results into 00600000. In Figure 13, 00780000 appears straight after 00600000. The HEX-values are converted to binary-values and make and addition in binary to show the same result, how the byte counter was structured. 00780000 in binary was equal to 11110000000000000000000 and we add 00180000 which was in binary equal to 11000000000000000000. The result in HEX-format as seen in Figure 13 should be 00900000 which was in binary equal to 100100000000000000000000. The same result in the calculation is demonstrated below in Figure 12:

$$\begin{array}{r} 11110000000000000000000 \\ +\quad 11000000000000000000 \\ \hline 1\,00100000000000000000000 \end{array}$$

Figure 12: Bytes addition for byte counter

- Byte-sequence: "YY YY YY YY" after the CPI Number as a Byte Counter
- Each new byte counter increases with 00180000 (HEX format)

For better understanding a set of numbered headers are collected, as seen in Figure 13, extracted from a CPI to show on row 1 that the underlined sequence on left side was the CPI Start. The sequence underlined on the right side was the first occurring CPI Number. Note that this CPI Number was exactly all the way down. On row 2, the underlined sequence on the left side was the Chirp Start followed by the CPI Number which was followed by the Byte Counter far to the right side. In between one byte counter and the following byte counter

there are large byte-sequences representing the radar data that we generate and modify to be suitable for a chosen scenario.

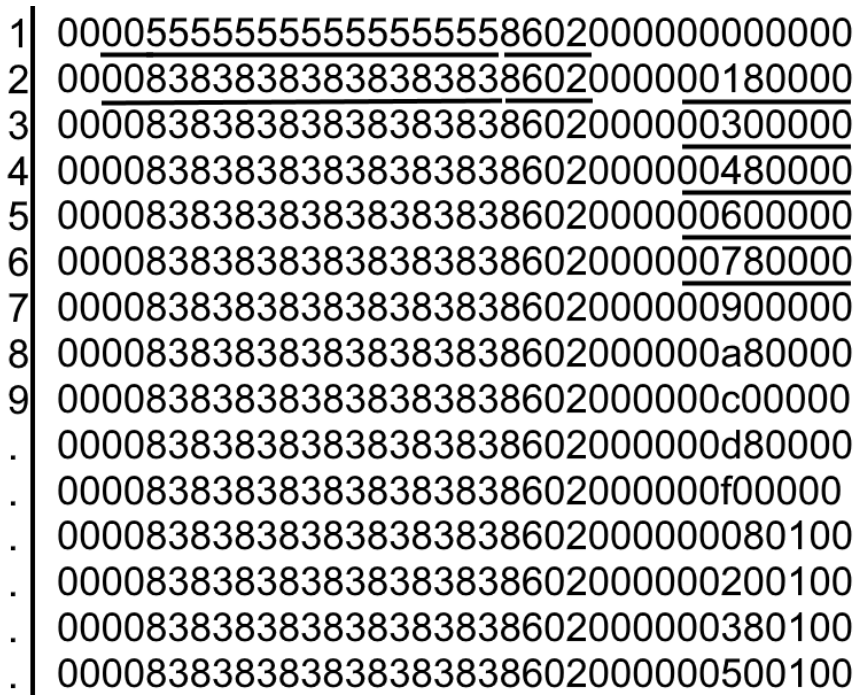


Figure 13: Output of collected headers with underlined CPI Start, Chirp Start, CPI Number and Byte Counter

In Figure 13 the file beginning is underlined in row 1, chirp start in row 2 followed by underlined byte counter. Nothing other than the headers in a CPI is included in this figure. Every row in the figure represents the beginning of a chirp and that header has a size of 18 bytes while the chirp itself has a size of 6162 *bytes*. What is missing in that figure between the rows are the remaining bytes of a chirp which a has a size of (6162 *bytes* − 18 *bytes* = 6144 *bytes*.

5.3.3 CPI Production

To produce a CPI, 3154944 *bytes* need to be inserted in HEX-format in a file saved as binary with the ending ".bin". The structure of bytes and relationships between the sequences of bytes inside that file are implemented according to available design recovery. Another Python software was developed in this thesis to create a binary file and fill it with bytes. The focus was on the chirp bytes in between the byte counters

If all the values are put to "00", the binary file processing with the client´s software ends up with failure.

In 5.3.2, we learn about a structure of 6144 bytes times 512 and by giving these bytes values above "00" detections start at different but increasing distances in X-axes.

If all values are put to "FF", the result was 66 detections on the X-axes. X-axes value was the distance from the gate to each object. In Figure 14 the detected X-axes values are presented, where the value of 26 meters is marked as an example.

512 byte sequences in the binary file with values of "FF" at all bytes give us 66 different values of distance in X-axes. The first X-axes value is 8,9 which is 8,9 meters from the gate. The last value is 55,7 which is 55,7 meters from the gate.


```
{"id":0,"position":{"x":8.935622236,"y":-0.852505431,"z":-5.074437717},"speed":{"x":0,"y":0,"z":0},
{"id":0,"position":{"x":9.786634428,"y":-0.933696477,"z":-5.557717813},"speed":{"x":0,"y":0,"z":0},
{"id":0,"position":{"x":10.63764579,"y":-1.014887444,"z":-6.040997439},"speed":{"x":0,"y":0,"z":0},
{"id":0,"position":{"x":11.48865799,"y":-1.09607849,"z":-6.524277534},"speed":{"x":0,"y":0,"z":0},"
{"id":0,"position":{"x":12.33966935,"y":-1.177269457,"z":-7.00755716},"speed":{"x":0,"y":0,"z":0},"
{"id":0,"position":{"x":13.19068072,"y":-1.258460424,"z":-7.490836786},"speed":{"x":0,"y":0,"z":0},
{"id":0,"position":{"x":14.04169291,"y":-1.33965147,"z":-7.974116882},"speed":{"x":0,"y":0,"z":0},"
{"id":0,"position":{"x":14.8927051,"y":-1.420842516,"z":-8.457396977},"speed":{"x":0,"y":0,"z":0},"
{"id":0,"position":{"x":15.74371564,"y":-1.502033404,"z":-8.940676134},"speed":{"x":0,"y":0,"z":0},
{"id":0,"position":{"x":16.59472784,"y":-1.58322445,"z":-9.42395623},"speed":{"x":0,"y":0,"z":0},"s
{"id":0,"position":{"x":17.44574003,"y":-1.664415496,"z":-9.907236325},"speed":{"x":0,"y":0,"z":0},
{"id":0,"position":{"x":18.29675057,"y":-1.745606384,"z":-10.39051548},"speed":{"x":0,"y":0,"z":0},
{"id":0,"position":{"x":19.14776276,"y":-1.82679743,"z":-10.87379558},"speed":{"x":0,"y":0,"z":0},"
{"id":0,"position":{"x":19.99877495,"y":-1.907988476,"z":-11.35707567},"speed":{"x":0,"y":0,"z":0},
{"id":0,"position":{"x":20.84978549,"y":-1.989179365,"z":-11.84035483},"speed":{"x":0,"y":0,"z":0},
{"id":0,"position":{"x":21.70079768,"y":-2.070370411,"z":-12.32363493},"speed":{"x":0,"y":0,"z":0},
{"id":0,"position":{"x":22.55180988,"y":-2.151561457,"z":-12.80691502},"speed":{"x":0,"y":0,"z":0},
{"id":0,"position":{"x":23.40282042,"y":-2.232752345,"z":-13.29019418},"speed":{"x":0,"y":0,"z":0},
{"id":0,"position":{"x":24.25383261,"y":-2.313943391,"z":-13.77347427},"speed":{"x":0,"y":0,"z":0},
{"id":0,"position":{"x":25.1048448,"y":-2.395134437,"z":-14.25675437},"speed":{"x":0,"y":0,"z":0},"
{"id":0,"position":{"x":25.95585534,"y":-2.476325325,"z":-14.74003353},"speed":{"x":0,"y":0,"z":0},
{"id":0,"position":{"x":26.80686753,"y":-2.557516371,"z":-15.22331362},"speed":{"x":0,"y":0,"z":0},
{"id":0,"position":{"x":27.65787973,"y":-2.638707417,"z":-15.70659372},"speed":{"x":0,"y":0,"z":0},
{"id":0,"position":{"x":28.50889192,"y":-2.719898463,"z":-16.18987381},"speed":{"x":0,"y":0,"z":0},
{"id":0,"position":{"x":29.35990411,"y":-2.801089509,"z":-16.67315391},"speed":{"x":0,"y":0,"z":0},
{"id":0,"position":{"x":30.210913,"y":-2.88228024,"z":-17.15643212},"speed":{"x":0,"y":0,"z":0},"si
{"id":0,"position":{"x":31.06192519,"y":-2.963471286,"z":-17.63971222},"speed":{"x":0,"y":0,"z":0},
{"id":0,"position":{"x":31.91293738,"y":-3.044662332,"z":-18.12299232},"speed":{"x":0,"y":0,"z":0},
{"id":0,"position":{"x":32.76394958,"y":-3.125853378,"z":-18.60627241},"speed":{"x":0,"y":0,"z":0},
{"id":0,"position":{"x":33.61496177,"y":-3.207044423,"z":-19.08955251},"speed":{"x":0,"y":0,"z":0},
{"id":0,"position":{"x":34.46597396,"y":-3.288235469,"z":-19.5728326},"speed":{"x":0,"y":0,"z":0},"
{"id":0,"position":{"x":35.31698615,"y":-3.369426515,"z":-20.0561127},"speed":{"x":0,"y":0,"z":0},"
{"id":0,"position":{"x":36.16799504,"y":-3.450617246,"z":-20.53939092},"speed":{"x":0,"y":0,"z":0},
{"id":0,"position":{"x":37.01900723,"y":-3.531808292,"z":-21.02267101},"speed":{"x":0,"y":0,"z":0},
{"id":0,"position":{"x":37.87001942,"y":-3.612999338,"z":-21.50595111},"speed":{"x":0,"y":0,"z":0},
{"id":0,"position":{"x":38.72103162,"y":-3.694190384,"z":-21.9892312},"speed":{"x":0,"y":0,"z":0},"
{"id":0,"position":{"x":39.57204381,"y":-3.77538143,"z":-22.4725113},"speed":{"x":0,"y":0,"z":0},"s
{"id":0,"position":{"x":40.423056,"y":-3.856572476,"z":-22.95579139},"speed":{"x":0,"y":0,"z":0},"s
{"id":0,"position":{"x":41.27406489,"y":-3.937763206,"z":-23.43906961},"speed":{"x":0,"y":0,"z":0},
{"id":0,"position":{"x":42.12507708,"y":-4.018954252,"z":-23.92234971},"speed":{"x":0,"y":0,"z":0},
{"id":0,"position":{"x":42.97608927,"y":-4.100145298,"z":-24.4056298},"speed":{"x":0,"y":0,"z":0},"
{"id":0,"position":{"x":43.82710147,"y":-4.181336344,"z":-24.8889099},"speed":{"x":0,"y":0,"z":0},"
{"id":0,"position":{"x":44.67811366,"y":-4.26252739,"z":-25.37218999},"speed":{"x":0,"y":0,"z":0},"
{"id":0,"position":{"x":45.52912585,"y":-4.343718436,"z":-25.85547009},"speed":{"x":0,"y":0,"z":0},
{"id":0,"position":{"x":46.38013474,"y":-4.424909167,"z":-26.33874831},"speed":{"x":0,"y":0,"z":0},
{"id":0,"position":{"x":47.23114693,"y":-4.506100213,"z":-26.8220284},"speed":{"x":0,"y":0,"z":0},"
{"id":0,"position":{"x":48.08215912,"y":-4.587291259,"z":-27.3053085},"speed":{"x":0,"y":0,"z":0},"
{"id":0,"position":{"x":48.93317131,"y":-4.668482305,"z":-27.78858859},"speed":{"x":0,"y":0,"z":0},
{"id":0,"position":{"x":49.78418351,"y":-4.749673351,"z":-28.27186869},"speed":{"x":0,"y":0,"z":0},
{"id":0,"position":{"x":50.6351957,"y":-4.830864397,"z":-28.75514878},"speed":{"x":0,"y":0,"z":0},"
{"id":0,"position":{"x":51.48620789,"y":-4.912055443,"z":-29.23842888},"speed":{"x":0,"y":0,"z":0},
{"id":0,"position":{"x":52.33721678,"y":-4.993246173,"z":-29.7217071},"speed":{"x":0,"y":0,"z":0},"
{"id":0,"position":{"x":53.18822897,"y":-5.074437219,"z":-30.20498719},"speed":{"x":0,"y":0,"z":0},
{"id":0,"position":{"x":54.03924116,"y":-5.155628265,"z":-30.68826729},"speed":{"x":0,"y":0,"z":0},
{"id":0,"position":{"x":54.89025336,"y":-5.236819311,"z":-31.17154738},"speed":{"x":0,"y":0,"z":0},
{"id":0,"position":{"x":55.74126555,"y":-5.318010357,"z":-31.65482748},"speed":{"x":0,"y":0,"z":0},
```


Figure 14: Print out of different distances that can be represented with our own generation of bytes

5.3.4 Trial and error

The discovered byte sequences in 5.4 are being put in the binary file, that represent an emulated CPI, as seen in Figure 15 using variable names:

- start

- chirpStart
- chirpToken

```
while(_loopTrigger):
    loopCount += 1
    print(loopCount)
    with open('TestB.bin', 'wb') as file:
        start = "\x00\x00\x55\x55\x55\x55\x55\x55\x55\x55"
        chirpStart = "\x00\x00\x83\x83\x83\x83\x83\x83\x83\x83"
        chirpToken = "\x6a\x0f\x00\x00"
        chirpLength = "00000000"
        startFlag = True
        lengthFlag = True
        count = 0
        length = 0

        while(count < 512):
```

Figure 15: Python code fragment for the beginning of a binary file

The reverse engineering on itself could not provide a solid design where Trial and Error has to be used to complete the design of a CPI. The next part of the code was to loop 512 times for the 512 byte sequences, followed by a byte counter as discovered in 5.3.2 and as seen in Figure 16 the variable name is "length" and it increases with 00180000 in HEX-format.

```
    file.write(toSave.encode("ISO-8859-1"))
    if(length > 0xFFFF00):
        length -= 0xFFFF00

    chirpLength = '{0:0{1}X}'.format(length, 8)
    count += 1
    length += 0x180000
```

Figure 16: Python code fragment for the byte counter

In between was a loop for the missing 512 byte sequences where, by looping through different number of groups, a structure of grouping these 512 bytes was found in 5 equal groups. In Figure 17 code fragment can be seen for the loop of these 512 byte sequences. At first each one of the 512 byte sequences was treated individually without any result on the detection, until each 100 sequences are treated at once in five different times.

```
start = "\x00\x00\x55\x55\x55\x55\x55\x55\x55\x55"
        chirpStart = "\x00\x00\x83\x83\x83\x83\x83\x83\x83\x83"
        chirpToken = "\x6a\x0f\x00\x00"
        ..............
        while(count < 512):
            ..........
            elif(count < 101):
                ..............
            elif(count < 201):
                ..............
            elif(count < 301):
                ..............
            elif(count < 401):
                ..............
            else:
                ..............
            file.write(toSave.encode("ISO-8859-1"))
```

Figure 17: Python code fragment for the 5 groups of the 512 byte sequences

A systematic change of the bytes was followed to find the association between the bytes sequences representing those detections. At first a lopp of 10 000 tries did not give the wanted result. 10 000 expanded to 20 000, 40 000 and then 60 000 tries (test numbers).

Each time a new binary file was created the value of the test number in the Python code was increased and was used as a reference for the certain structure of the 512 byte sequences. It was also used as a factor when changing from one byte to another. The same test number was used as a reference for the processing result of detection. Thus, certain detection results was tracked back to a certain structure of the 512 byte sequences. By hard-coding the value of the variable into the Python code with one specific test number to recognize the one specific result of detections that is wanted.

- The first 10 000 tries gave as best a distance up to 11,5 meters
- Tries between 10 000 & 20 000 gave as best a distance of up to 14 meters
- Tries between 20 000 & 30 000 gave as best a distance of up to 16,4 meters
- Tries between 30 000 & 40 000 gave as best a distance of up to 19,3 meters

From a total of 60 000, on the 42218th try the result of a distance of 21 meters in X-axes was obtained. In the Python code, the variable "adder_str" was set to a value of the loop-position converted in to binary-format.

- 42218 in decimal for was converted to 1010010011101010 in binary

Variable "count" represents the current Chirp and gets increased for the next Chirp. Variable "toSave" represents the content of each Chirp added to the content of previous Chirp was finally representing the whole content of a binary file.

To summarize, following the architecture in 8 our Pyhton program takes an input (distance) where a CPI was generated and stored as a binary file. An UDP client shall establish connection with the UDP Server inside the client processing software. Upon successful connection the client sends the content of the stored binary file and waits for the processing result coming back in JSON format. The obtained detections of our CPI:s should be of the JSON-format as per the first requirement in 5.1. The second requirement was of measurable nature which impose a condition on the prototype of having to make sure that five CPI:s can be sent for processing with their results in JSON-format ready within one single second.

Detection results follow specification given bu the client as senn below in Listing 1 where the result should be placed inside "tracks".

Listing 1: Detection data specification

```
{
    "apiVersion": "0.1",

    "data":
    {
        "msgType": "reply",

        "status": "running",

        "tracks":[{}]
    }
}
```

5.4 Development of the system

Building the prototype following the architecture in Figure 8 and the design in 5.3 the prototype, our Pyhton program, was taking the distance as a parmeter measured in meters and represented with a digit responding to that certain distance. 21 meters was represented with the digit 42218.

A CPI was stored as a binary file in a folder for CPI:s and UDP connection was established using the port number given by the client processing software. A simple system overview is presented in Figure 18

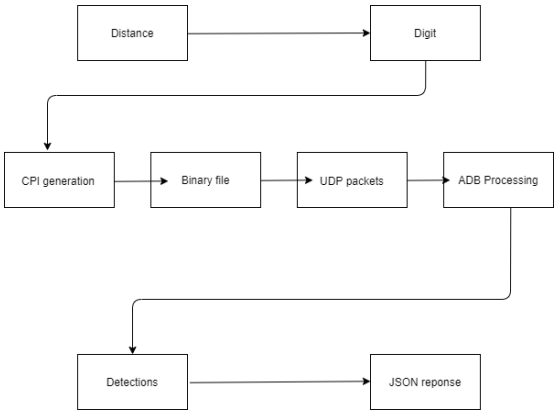


Figure 18: System overview of the prototype stages

5.5 Evaluation

As stated in 4.5, the evaluation was performed with our generated CPI that was processed with the software from the client where the result was displayed in JSON-format. From 5.3.4 there was a test number with the value of 42218 which when put into the Python code along with the other byte tokens needed a CPI was generated to send for processing. The stages in Figure 18 were followed in the order presented in the figure. The result is seen in Appendix A as detections with X-axes values from 21 meters up to 28 meters.

6 Discussion

This chapter presents a discussion about the research method and the results of this thesis.

6.1 Iterative approach

The chosen method in Chapter 4 of Nunamaker is based on the iterative approach within that method. It was used several times to overcome failures. It was also used for the reverse engineering of the CPIs to find special byte sequences and special structure of byte sequences.

6.2 Results discussion

The most crucial part of this thesis which is also reflected in the RQ is how to use reverse engineering on binary files with minimum specifications. The choice has to be made as in for what purpose RE is used, either for re-documentation or design recovery, mentioned in 3.3 from [34]. The design of CPI:s needs to be recovered to generate CPI:s.

The lack of specifications and lack of CPI structure design made us to start with the RE following into the steps of previous researches in [33] [34] [35] [36] [37] [38] [39] [40] [41] [42] [43] [44] [45] [46] [47] [48] [49] [50] surveyed in 3.1, by structuring the process of RE into three stages:

- Observe and analyze
- Disassemble
- Decompile

The binary content is observed and analyzed in the provided CPI:s and could outline a certain pattern of binary values stored in those binary files at the beginning and at the end of each file. Disassembly is performed on all the 3764 provided files to confirm those file beginnings and endings. In [36] that disassembly is used to identify clearly outlined segments and also less clearly outlined foreign segments, foreign to the obvious segments. Although in [36] they are provided with a specification when they are disassembling, something of great help for any disassembly, binary files have to be dissembled as they were provided without any specification. A divide and conquer approach is performed to create a specification which is followed later on for assembling a binary file. The data between the file beginning and the file ending is considered as one entity of binary values. When the huge size in bytes of that entity is realized we had to switch from manual process to automatic process when performing the Assembly of that entity.

Automatic processes have been proposed by researchers [53] [54] [55] [56] whereas for us it lead us to dividing that huge entity into smaller entities. The expression divide and conquer is used because moving forward from one single file to one

file beginning, one file ending and one huge entity in between. The beginning and the ending is hereby conquered and put aside. The huge entity is further dissembled into chirp entities. Furthermore for each chirp entity smaller entities were dissembled into chirp beginning, CPI number and byte counter.

Through the stage of RE an iterative approach is followed which helped to understand the structure and the algorithm behind the byte counter. The RE is performed on a huge scale, meaning dissembling all 3764 files, to leave no room for chances or randomly found associations between the the discovered byte sequences/entities,

In any binary file with 3154944 *bytes* the beginning is a byte sequence of "00 00 55 55 55 55 55 55 55 55". 6162 bytes represents a chirp and a whole file contains 512 chirps. Each chirp byte sequence begins with "00 00 83 83 83 83 83 83 83 83", the chirp start, followed by a CPI number special for that file. Also, common for all chirps is to end with a systematically increasing byte counter as mentioned earlier and explained in 5.3.2. These associations and general values are not bound or restricted to certain detection results, rather general for any CPI to be created/generated.

The most important part of this thesis is to find out if RE could lead us the whole way through to a special/certain structure of byte sequences bound to certain detection results. We know now that RE was not enough for that purpose and we had to extend with a Trial and Error approach for the chirp byte sequences. During the process of Trial and error process we achieved detection results starting in a distance of 21 meters in X-axes and ending in a distance of 28 meters which gives a length of 7 meters for that object. It was achieved by iterating while increasing binary values in the raw data up to 60 000 times. Before that a disassembly of 37654 binary files was performed. The iteration included how many chirp bytes sequences of total 512, was given values simultaneously starting with each one separately. The process is repeated 60 000 times before changing to each two of the chirps byte sequences repeated 60 000 times followed by a change to each three, four and so on for the sake of being systematic. The result was to set values to each hundred of chirps byte sequences simultaneously before moving on to the next hundred with and to use different values on those up to 512 sequences and repeated 60 000 times.

The lack of specification of the provided CPI:s impacted on the results of the RE, why we had to move on to Trial and error approach. Analog radar signal is processed and and converted to a digital one. Neither the software nor the specification for that conversions were available for us. For future work further Trial and error processes can be performed for further defections. Also, as we did see in related researches that many processes/cases of RE depend on existing documentation of a software, the software itself or design specifications. For Hardware RE where the components were visible during the RE, the RE were successful. In our thesis the radar signal were converted to digital and without documentation, conversion software or design specifications we end up with invisible entities/components where we found the Trial and Error approach to

be appropriate approach.

7 Conclusion and future work

7.1 Answering the research question

The research question in this thesis as seen in 1.4 is answered below:

RQ: How can Reverse Engineering of real radar data, help to generate imitated data and function as if it was real radar data?

Answer: The reverse engineering could not on itself help to generate a CPI and therefor had to be completed with a stage of trial and error methodology. RE did lead us during the design stage onto how the binary file should be structured. From an entity of 3154944 *bytes* in each one of the 2764 provided binary files, further smaller entities were identified through reversing backwards. With reverse engineering of real radar data, as CPIs in binary files, several key elements in those files are discovered. The files are observed and disassembled to help us to prepare the initial object model, files size and file type. Also, it helped us to determine essential values and discover generalizations and associations. RE could not be performed on the raw data entity for each one of the 512 chirps because of failure in generalization and association discovery.

Trial and Error were used to try replicating those chirp entities and by doing that 60 000 times a CPI was generated representing a detection of a 21 meters distance. The discovered tokens from RE together with data from trial and error-stage were used to create/generate a binary file in such a way as if it was real radar data because processing that data with the same software used to process CPIs from real radar data gives detections of an object.

7.2 Hypothesis

The hypothesis in 1.4 for this thesis is:

Hypothesis: It is possible to generate data so that a VDGS system thinks it is real radar data.

The proposed hypothesis is proven by finding relationships between variables in those binary files using reverse engineering as seen in 7.1. The outcome of detections is shown in Appendix A.

7.3 Future work

Since the radar will be used at airports when the aeroplanes land onto the runway to continue their way through the apron area and finally park at different gates, that means different distances of away from the gates. In this thesis a certain distance of 21 meters was targeted as the aim of our research which now can be used as a basis to target other distances and in such a way reflect several different positions of an aeroplane away from the gate. Despite the failure of

using reverse engineering on its own in the research to produce/emulate radar data files, this research still has its contribution to the concerned science society such as:

- Reverse engineering with limitations is doable without previous target object design/documentation
- Disassembly & analyzes with limitations of the target object can be used to produce at least a skeleton of a replica object
  - Prepare initial object model
  - Determine essential values
  - Discover generalizations
- Discovery of associations between the smallest object components(raw radar data) is not possible with RE on its own specially without documentation on how radar data is converted from analog till digital and how it is packaged into certain byte-sequences.
  - Complementing with Trial & Error process, some associations is discovered to produce at least one data file
  - Statistics of Trial & Error: one replicated data file required production of 60 000 unwanted files
- The main contribution of this research: when provided with the same material used in this thesis, if it is the radar data files, the lacking documentation and so on, Reverse engineering without trial & error is not enough and Trial & Error without reverse engineering isn't either enough.

Therefor, as future work might be found by targeting other and further distances from the gate followed by a simulated, smooth and continuous movement of an aeroplane in a another further future work.

References

[1] Eggebeen, J. (2007). Airport age: Architecture and modernity in America. City University of New York.

[2] Higham, R. D. (2003). One Hundred Years of Air Power and Aviation (No. 5). Texas A&M University Press.

[3] Robert B. Fairbanks, "Airports," in Encyclopedia of Urban America, vol. I, ed. Neil Larry Shumsky, 1998. In comparison, "Amusement Parks" has more than three pages, "Correctional Institutions" and "Railroad Stations" each have two pages.

[4] Ha, H. K., Wan, Y., Yoshida, Y., & Zhang, A. (2013). Airline market structure and airport efficiency: Evidence from major Northeast Asian airports. Journal of Air Transport Management, 33, 32-42.

[5] ADB Safegate. (2020). About / Corporate / Our focus. Retrieved (February 12, 2020) from https://adbsafegate.com/about/corporate/our-focus/

[6] Wikipedia.org (2020). Airport. Retrieved (April 12, 2020) from https://commons.wikimedia.org/wiki/File:Airport_infrastructure.png

[7] Voicemedia (2020). Dublin Voice. Retrieved (February 12, 2020) from https://www.voicemedia.ie/dublin-voice/airports-new-docking-will-cut-emissions/

[8] thyssenkrupp (2020). Airport Solutions. Retrieved (February 12, 2020) from https://www.thyssenkrupp-elevator.com/airport-solutions/products/intelligent-gate-systems/vdgs/

[9] FMT (2020). Airport Systems. Retrieved (February 12, 2020) from https://www.fmt.se/airport/

[10] ADB Safegate. (2020). Gate – making turnarounds faster and more predictable (February 12, 2020) from https://adbsafegate.com/gate/

[11] Thuan, N. H., Drechsler, A., & Antunes, P. (2019). Construction of design science research questions. Communications of the Association for Information Systems, 44(1), 332-363.

[12] Prasad, S., Rao, A., & Rehani, E. (2001). Developing hypothesis and research questions. 500 Research Methods, 1-30.

[13] RaillyNews. (2020). Airports (February 12, 2020) from https://www.raillynews.com/guncel/karayolu/havalimanlari/

[14] ICAO, C. (2013). "Phase of Flight, Definitions and Usage Notes. CAST/ICAO Common Taxonomy Team, http://www. intlaviationstandards. org/-Documents/PhaseofFlightDefinitions. pdf, Tech. Rep.

[15] Chaolong, T., Bo, C., Shuai, Z., & Dalin, Z. (2015, December). Distributed networked radar detection scheme. In 2015 4th International Conference on

Computer Science and Network Technology (ICCSNT) (Vol. 1, pp. 416-420). IEEE.

[16] ADB Safegate. (2020). A-VDGS: Safedock X. Retrieved (February 12, 2020) from https://adbsafegate.com/product-center/gate/?prod=safedock-x

[17] Youtube (2020). 12Bpilot's SODE VDGS. Retrieved (February 12, 2020) from https://www.youtube.com/watch?v=EAQXM1ZFK4k

[18] dSPACE (2020). Realistic simulation of radar signals. Retrieved (February 16, 2020) from https://www.dspace.com/en/pub/home/products/sw/sensor_sim/rada_module.cfm

[19] Nikodym, T. (2010). Ray tracing algorithm for interactive applications. BSc, Czech Technical University, Prague, Czech Republic.

[20] Mahafza, B. R. (2017). Introduction to radar analysis. Chapman and Hall/CRC.

[21] Girma, M. (2019). Concepts for Short Range Millimeter-wave Miniaturized Radar Systems with Built-in Self-Test (Vol. 92). KIT Scientific Publishing.

[22] Radar Basics(2020). The Radar Range Equation. Retrieved (February 16, 2020) from https://www.radartutorial.eu/01.basics/The%20Radar%20Range%20Equation.en.html

[23] Meikle, H. (2008). Modern radar systems. Artech House.

[24] Lee, H., Kim, T. H., Choi, J. W., & Choi, S. (2015, April). Chirp signal-based aerial acoustic communication for smart devices. In 2015 IEEE Conference on Computer Communications (INFOCOM) (pp. 2407-2415). IEEE.

[25] Smith, S. (2013). Digital signal processing: a practical guide for engineers and scientists. Elsevier.

[26] Kim, S. (2015). Bio-inspired engineered sonar systems based on the understanding of bat echolocation. In Biomimetic Technologies (pp. 141-160). Woodhead Publishing.

[27] Nakamura, R. Y. M., Pereira, L. A. M., Rodrigues, D., Costa, K. A. P., Papa, J. P., & Yang, X. S. (2013). Binary bat algorithm for feature selection. In Swarm Intelligence and Bio-Inspired Computation (pp. 225-237). Elsevier.

[28] embien - Empowering and Innovation(2019). INTRODUCTION TO LORA TECHNOLOGY (February 16, 2020) from https://www.embien.com/blog/introduction-to-lora-technology/

[29] Texas Instruments(2017). The fundamentals of millimeter wave sensors (February 24, 2020) from http://www.ti.com/lit/wp/spyy005/spyy005.pdf

[30] Texas Instruments. Introduction to mmwave Sensing: FMCW Radars, Sandeep Rao, (February 24, 2020) from

https://training.ti.com/sites/default/files/docs/mmwaveSensing-FMCW-offlineviewing_0.pdf

[31] Polytechnic Hub(2018). What do you mean by ADC (Analog to digital converter)? (February 24, 2020) from https://www.polytechnichub.com/mean-adc-analog-digital-converter/

[32] Texas Instruments(2018). Application Report: Mmwave Radar Device ADC Raw Data Capture (February 24, 2020) from http://www.ti.com/lit/an/swra581b/swra581b.pdf

[33] Rekoff, M. G. (1985). On reverse engineering. IEEE Transactions on systems, man, and cybernetics, (2), 244-252.

[34] Chikofsky, E. J., & Cross, J. H. (1990). Reverse engineering and design recovery: A taxonomy. IEEE software, 7(1), 13-17.

[35] Fyrbiak, M., Strauß, S., Kison, C., Wallat, S., Elson, M., Rummel, N., & Paar, C. (2017, July). Hardware reverse engineering: Overview and open challenges. In 2017 IEEE 2nd International Verification and Security Workshop (IVSW) (pp. 88-94). IEEE.

[36] Premerlani, W. J., & Blaha, M. R. (1993, May). An approach for reverse engineering of relational databases. In [1993] Proceedings Working Conference on Reverse Engineering (pp. 151-160). IEEE.

[37] Wilson, J. O., & Rosen, D. (2007, January). Systematic reverse engineering of biological systems. In International Design Engineering Technical Conferences and Computers and Information in Engineering Conference (Vol. 48043, pp. 69-78).

[38] Nagel, J. K., Stone, R. B., & McAdams, D. A. (2010, January). An engineering-to-biology thesaurus for engineering design. In International Design Engineering Technical Conferences and Computers and Information in Engineering Conference (Vol. 44137, pp. 117-128).

[39] Otto, K., & Wood, K. (1996, August). A reverse engineering and redesign methodology for product evolution. In Proceedings of the 1996 ASME Design Theory and Methodology Conference (Vol. 96).

[40] Otto, K. N. (2003). Product design: techniques in reverse engineering and new product development. Tsinghua University Press Co., Ltd.

[41] Harston, S. P., & Mattson, C. A. (2010). Metrics for evaluating the barrier and time to reverse engineer a product. Journal of Mechanical Design, 132(4).

[42] Foglesong, T., Arlitt, R., Stone, R., & Parmigiani, J. (2015, November). Model-based design to support complex systems implementation as a result of reverse engineering. In 2015 International Conference on Complex Systems Engineering (ICCSE) (pp. 1-6). IEEE.

[43] Villaverde, A. F., & Banga, J. R. (2013). Reverse engineering and identification in systems biology.

[44] Villaverde, A. F., & Banga, J. R. (2014). Reverse engineering and identification in systems biology: strategies, perspectives and challenges. Journal of the Royal Society Interface, 11(91), 20130505.

[45] Siirtola, M. (2020). Identifying means to improve the manufacturability of marine feeder-booster fuel supply units using competitive benchmarking, reverse engineering and thematic interview.

[46] Nelson, M. L. (2005). A survey of reverse engineering and program comprehension. arXiv preprint cs/0503068.

[47] Hall, P. A. (1992). Software reuse and reverse engineering in practice. Chapman & Hall, Ltd..

[48] Eilam, E. (2011). Reversing: secrets of reverse engineering. John Wiley & Sons.

[49] Peikari, C., & Chuvakin, A. (2004). Security Warrior: Know Your Enemy. " O'Reilly Media, Inc.".

[50] Shahbaz, M. (2012). Reverse Engineering and Testing of Black-Box Software Components. LAP Lambert Academic Publishing.

[51] Lee, N. (2013). Counterterrorism and cybersecurity. Springer Science Business Media, New York.

[52] Hertel, C. "Samba: An Introduction," The Open Group, 6/10/1999, retrieved 9/3/2001 from the World Wide Web: <http://usl.samba.org/samba/docs/SambaIntro.html>, 1996. Google Scholar

[53] Cui, W., Kannan, J., & Wang, H. J. (2007, August). Discoverer: Automatic Protocol Reverse Engineering from Network Traces. In USENIX Security Symposium (pp. 1-14).

[54] Cui, W., Peinado, M., Chen, K., Wang, H. J., & Irun-Briz, L. (2008, October). Tupni: Automatic reverse engineering of input formats. In Proceedings of the 15th ACM conference on Computer and communications security (pp. 391-402).

[55] Cui, W., Peinado, M., Chen, K., Wang, J. H., & Irun-Briz, L. (2015). U.S. Patent No. 8,935,677. Washington, DC: U.S. Patent and Trademark Office.

[56] Comparetti, P. M., Wondracek, G., Kruegel, C., & Kirda, E. (2009, May). Prospex: Protocol specification extraction. In 2009 30th IEEE Symposium on Security and Privacy (pp. 110-125). IEEE.

[57] W3SCHOOL.COM(2020). JSON - Introduction (February 24, 2020) from https://www.w3schools.com/js/js_json_intro.asp

[58] Basu, A. (2015). Software quality assurance, testing and metrics. PHI Learning Pvt. Ltd..

[59] Engström, E., Runeson, P., & Ljung, A. (2011, March). Improving Regression Testing Transparency and Efficiency with History-Based Prioritization–An Industrial Case Study. In 2011 Fourth IEEE International Conference on Software Testing, Verification and Validation (pp. 367-376). IEEE.

[60] Lutz, S., Erhart, C., Walte, T., & Weigel, R. (2014, October). Target simulator concept for chirp modulated 77 GHz automotive radar sensors. In 2014 11th European Radar Conference (pp. 65-68). IEEE.

[61] Peffers, K., Tuunanen, T., Rothenberger, M. A., & Chatterjee, S. (2007). A design science research methodology for information systems research. Journal of management information systems, 24(3), 45-77.

[62] Keen, P. G. (1987). MIS research: Current status, trends and needs. Information systems education: Recommendations and implementation, 1, 13.

[63] Nunamaker Jr, J. F., Chen, M., & Purdin, T. D. (1990). Systems development in information systems research. Journal of management information systems, 7(3), 89-106.

[64] Bailey, K. D. Methods of social research. 1982. Nova Iorque, EUA: The Free Press, 2.

[65] Blalock, A. B., and Blalock, H. M.. Jr. Introduction to Social Research, second edition. Englewood Cliffs, NJ: Prentice-Hall, 1982.

[66] Shaw, R. S., Tsao, C. F., & Wu, P. W. (2012). A study of the application of ontology to an FAQ automatic classification system. Expert Systems with Applications, 39(14), 11593-11606.

[67] Richardson, T., Stafford-Fraser, Q., Wood, K. R., & Hopper, A. (1998). Virtual network computing. IEEE Internet Computing, 2(1), 33-38.

[68] Hu, Y., & Neamtiu, I. (2016, May). VALERA: an effective and efficient record-and-replay tool for android. In Proceedings of the International Conference on Mobile Software Engineering and Systems (pp. 285-286).

Appendix

A Detections

Detections in Json-format as per the client template:


```
{
    "apiVersion": "0.1",
    "data":
    {
        "msgType": "reply",
        "status": "running",
        "tracks": [ {
        "id":0,
        "position":
        {"x":21.06208476,"y":0.415620418,"z":-1.41311005},
        "speed":{"x":0,"y":0,"z":0},
        "size":{"x":0,"y":0,"z":0},
        "quality": {"SNR":74.8240509} }, {
        "id": 0,"position":
        {"x":22.04171723,"y":0.434951612,"z":-1.478836141},
        "speed":{"x":0,"y":0,"z":0}, "size":{"x":0,"y":0,"z":0},
        "quality":{"SNR":74.01108551} }, {
        "id":0,"position":
        {"x":23.0213497,"y":0.454282807,"z":-1.544562231},
        "speed":{"x":0,"y":0,"z":0},
        "size":{"x":0,"y":0,"z":0},
        "quality":{"SNR":73.24017334}},{
        "id":0,"position":
        {"x":24.00098027,"y":0.473613964,"z":-1.610288194},
        "speed":{"x":0,"y":0,"z":0},
        "size":{"x":0,"y":0,"z":0},
        "quality":{"SNR":72.48933411}},{
        "id":0,"position":
        {"x":24.98061274,"y":0.492945158,"z":-1.676014284},
        "speed":{"x":0,"y":0,"z":0},"size":{"x":0,"y":0,"z":0},
        "quality":{"SNR":71.78722382}},{
        "id":0,"position":
        {"x":25.96024521,"y":0.512276353,"z":-1.741740375},
        "speed":{"x":0,"y":0,"z":0},
        "size":{"x":0,"y":0,"z":0},
        "quality":{"SNR":71.0914917}},{
        "id":0,"position":
        {"x":26.93987577,"y":0.53160751,"z":-1.807466337},
        "speed":{"x":0,"y":0,"z":0},
```

```
        "size":{"x":0,"y":0,"z":0},
        "quality":{"SNR":70.42287445}},{
        "id":0,"position":
        {"x":27.91950824,"y":0.550938704,"z":-1.873192428},
        "speed":{"x":0,"y":0,"z":0},
        "size":{"x":0,"y":0,"z":0},
        "quality":{"SNR":69.80632019}},{
        "id":0,"position":{"x":28.89914071 .....
        ......
        "a break here save place" {}}]}}


        }]
    }
}
```